

新加坡租房对话 Agent 实施计划与思路

从数据勘探到部署上线的七步拆解

参考日期 2026-07-28 数据集 502 条合成房源 · 10 个测试 session 预算工时 ≈ 20h / 2.5 天

0 一页纸总览

作业本身是开放式的，真正的难点不在“能不能做出一个聊天框”，而在**三件事必须同时成立**：数据是脏的但回答必须可信、需求是模糊且会变的但状态必须准确、有些请求（敏感偏好、自相矛盾的条件）没有标准答案但必须给出有理有据的处理。下面七步就是围绕这三件事排的。

#	阶段	核心产出	工时
1	需求解码与数据勘探	能力清单、数据画像、约束可行性表	2h
2	离线数据清洗与质量汇报	<code>listings.clean.json</code> + 质量报告	2h
3	检索与打分引擎（无 LLM）	确定性 <code>searchListings()</code> + 单测	3h
4	对话状态与需求抽取	<code>RequirementState</code> + slot filling	3h
5	Agent 编排与 grounding	tool-calling 回路、推荐理由生成	3h
6	前端 Demo	对话界面 + 房源卡片 + 数据质量页	3h
7	难 case 打磨 · 部署 · README	10 个 session 全跑通、上线、文档	4h

贯穿全程的三条原则

- ① **确定性的事不交给 LLM**。清洗、过滤、排序、算距离全部用代码，可测试、可复现、零成本。LLM 只负责两件它真正擅长的事：把模糊的自然语言变成结构化需求，把结构化结果讲成人话。
- ② **模型只能看见清洗后的结构化字段**。房源以 JSON 卡片形式经 tool 返回，模型拿不到原始脏数据、也拿不到库外信息——grounding 不是靠 prompt 里写“不要编造”，是靠物理上够不着。
- ③ **不确定性要显式暴露，而不是抹平**。缺面积就说缺面积，通勤是估算就标注估算，房东有国籍限制就提前告知。宁可说“我不知道”，不能说错。

1 需求解码与数据勘探

2h

动手写代码之前先做两件事，这两小时能省掉后面一整天的返工。

1.1 把评分维度翻译成可执行动作

作业末尾的四个考察维度不是抽象的，每一条都对应具体的、可以在 demo 里被看见的东西：

考察维度	翻译成"我要做出什么"
AI Agent 系统设计	可见的对话状态面板（用户能看到 agent 当前理解的需求）；tool-calling 而非 prompt 拼接；每条推荐理由都能追溯到具体字段
产品设计与思考	冷启动时的引导式提问；无匹配时的 trade-off 呈现；敏感请求的处理路径
工程能力	清洗管线 + 单元测试 + 数据质量页面；一条命令可复现
落地与表达	网站真的能用（不报错、有 loading、手机能开）；README 讲清取舍而不是罗列功能

1.2 先读 10 个 session，再看数据

10 个测试 session 实际上是一份**反向的需求文档**——它们精确地定义了 agent 必须具备哪些能力。逐个读完可以提炼出六项硬能力：

- **通勤计算**（session-01 到 CBD、02 到 Bukit Timah 国际学校、04 到 NUS Kent Ridge）
- **本地术语解释**（session-03：HDB vs Condo、master room、agent fee）
- **无匹配时的 trade-off 引导**（session-04、05、06）
- **单房源深度问答且不能编**（session-07）
- **敏感请求的有原则处理**（session-08）
- **状态增量更新而非重来**（session-09）与**主动提问**（session-10）

1.3 用数据验证这些 case 到底有多难

这一步很关键：**先算清楚每个 session 的约束在库里到底有几条命中**，才知道 agent 该说什么。实测结果：

Session	约束	完全命中	含义
05 中心+大+便宜	Orchard · 2 房 · ≥900sqft · <\$3,000	0	Orchard 全区仅 10 条，且无论什么价格都没有 2 房 ≥900sqft——必须谈取舍，不能硬凑
06 宠物+做饭+短租	petFriendly · cookingAllowed · 租期 ≤6 月	21	其中仅 7 条是单间；能满足但可选面窄，要提示"放宽哪一条能多出多少"
04 NUS 学生	单间 · ≤\$850	13	清洗后全岛只剩 13 条，且没有一条在 Kent Ridge 附近；放宽到 \$1,000 变 29 条

这一步顺手抓到了一个说服力很强的论据

"单间 ≤\$850" 的原始命中是 18 条，其中 5 条是脏数据（2 条租金为 0、3 条掉了一位数字：\$185 / \$245 / \$270 的公寓单间）。也就是说**如果不清洗，会有 28% 的假房源被当成"便宜好选择"推荐给一个预算紧张的学生**。这个数字直接写进 README，比任何"我很重视数据质量"的表述都有力。

产出 能力清单（6 项）· 数据画像 · 每个 session 的可行性基线数字

一次性离线脚本，产出静态文件，运行时不再碰原始数据。这样清洗成本为零、结果可复现、可以写测试。

2.1 实测的脏数据全貌

数据字典说"约 60 行 + 2 重复"，实跑统计后每一类恰好 6 行，全部定位到位：

类型	实际值	数量	处理
租金为字符串	"2,050" · "\$1200" · "4400/mo" · "\$5550 negotiable"	6	解析为数字，并保留"可议价"信号
入住日为文本	"Immediate"×2 · "immediate"×3 · "ASAP"	6	转小写后匹配 → 参考日 + <code>isImmediate</code>
地铁信息缺失	<code>nearestMrt = null</code>	6	按坐标估算最近站，标 <code>inferred</code>
邮区缺失	<code>district = ""</code> 或 <code>null</code>	6	由 area 映射回填，标 <code>inferred</code>
面积缺失	<code>sizeSqft = null</code>	6	不猜；用户要求面积时不参与筛选
押金为字符串	"1 month"	6	正则 → <code>{ months: 1 }</code>
区域名胜	" toa payoh " 等 6 个	6	trim + 别名表规范化
字段矛盾	<code>listingType=room</code> 但 <code>bedrooms=2/3</code>	6	以 <code>listingType</code> 为准，记录矛盾
描述混乱/混合语言	—	≈6	原文仅用于展示，不进匹配文本
租金异常	0 ×2 · 185 · 245 · 270 · 62500	6	移出推荐池，见 2.2
完全重复	SG0121 · SG0474	2	按 id 去重 + 全字段哈希双重校验

2.2 异常租金：分组基准，而不是全局分位数

全局砍尾会误伤——有地住宅整租的中位数本来就是 \$15,600，最高 \$24,350 完全合法。正确做法是按 (`listingType`, `propertyType`) 分组求中位数，再看比值：

分组	n	中位数	p10	p90
单间 / HDB	88	1,200	800	1,750
单间 / 公寓	112	1,950	1,350	2,700
整租 / HDB	116	3,400	2,650	4,550
整租 / 公寓	107	5,800	3,450	8,650
整租 / 服务式公寓	39	6,300	4,000	10,050
整租 / 有地住宅	18	15,600	9,000	21,550

阈值 `ratio < 0.25` 或 `> 4` 精确命中这 6 条、零误伤： $\$185/\$1,950 = 0.09$ 、 $\$62,500/\$5,800 = 10.8$ 被抓；而 \$650 的 HDB 单间 (0.54) 和 \$24,350 的有地住宅 (1.56) 安全通过。

抓到之后：标记，不改写

这四条掉零/多零的数据乘除 10 之后都精确落回合理区间 (185→1,850, 62,500→6,250)，修复几乎是确定的。但我
不自动改写价格字段。租房产品里价格错误是伤害最大的一类错误——用户按 \$185 约看房、到场发现是 \$1,850，信任就没了；而且 agent 一旦拿推断值去讲推荐理由，就违背了 grounding 原则。

所以拆成两件事：① 原始值原样保留，不覆写；② 修复建议写进 suggestedFix，只进报告、不进业务字段，改不改由人决定。同时 isRecommendable: false 把它移出推荐池。

2.3 规范化的具体算法

区域名：闭集 + 多数票，不手写映射表

不写别名表，让映射从数据自己长出来：① 生成归一键 (trim → 折叠连续空格 → 转小写)；② 按归一键分组，取
组内出现频次最高的原始拼写作为 canonical；③ 组内全部改写。实跑验证：52 个 distinct 收敛到 46 个，6 个
脏值全部找到干净对照，零歧义。

归一键	组内分布	canonical
eunos	Eunos ×14 · " eunos " ×1	Eunos
marine parade	Marine Parade ×14 · " marine parade " ×1	Marine Parade
ang mo kio	Ang Mo Kio ×14 · " ang mo kio " ×1	Ang Mo Kio
west coast	West Coast ×11 · " west coast " ×1	West Coast
toa payoh	Toa Payoh ×8 · " toa payoh " ×1	Toa Payoh
choa chu kang	Choa Chu Kang ×6 · " choa chu kang " ×1	Choa Chu Kang

绝对不要用 title-case 做规范化

语料里有 one-north (站名的正确写法就是全小写) 和 Harbourfront，无脑 title-case 会把它们改坏。多数票法
只在数据内部找共识、不引入任何外部规则，天然避开这个坑。

配套断言：若某个归一键的组里只有脏写法、没有干净对照，脚本直接报错退出（当前数据 0 个孤儿，但库更新后可能
出现——那时该由人看一眼，而不是让脚本猜）。

租金：白名单模式，解析失败即报错

关键取舍是不写"万能"正则——宽松的 \d+ 提取器遇到没预料的格式会静默返回一个错误数字，比崩掉危险得
多。

1. 已是 number

→ 直接进入范围校验
2. 是 string

→ 依次匹配已知模式（有序，先具体后宽泛）

^S?\\$?\s*([\d,]+)\$

→ "S\$1200" "2,050" "6,300"

^S?\\$?\s*([\d,]+)\s*/\s*(month)?\$

→ "4400/mo"

^S?\\$?\s*([\d,]+)\s+negotiable\$

→ "\$5550 negotiable" ⇒ negotiable = true
3. 去千分位逗号

→ parseInt → 范围校验（见 2.2）
4. 全部不匹配

→ 抛错，原始值进 issue, severity = block, 脚本非零退出

- negotiable 是业务信号不是噪声——用户问"能不能砍价"时，agent 就有据可依。
- 解析成功 ≠ 值合理，必须再过一遍 2.2 的分组比值校验。

- 收尾断言：502 条 100% 解析成功、0 条走 fallback。出现新格式 CI 就红——这才是白名单的意义。
- 押金同理：只匹配 `^(\d+)\s*months?$`，其余报错。

2.4 三档严重度 = 三种产品行为

不静默丢弃任何一行，而是给每条打上 `dataQuality` 标记，让数据问题在**产品层面**有确定的表现：

严重度	触发	Agent 行为
block	租金异常、重复 id	移出推荐池，用户永远看不到，进入人工队列
warn	面积缺失、邮区靠推断、字段矛盾	照常推荐，但回答时必须披露："这套未提供面积，我是按其他条件匹配的"
info	区域名空格、租金格式、押金字符串	自动规范化，不影响任何人，仅留审计痕迹

2.5 通用的数据质量汇报通道

每条清洗规则不各自 `console.log`，而是统一往一个 collector 上报结构化 issue。这让"我写了个清洗脚本"升级成"我把数据接入当成一个有运维回路的流程"：

```
type DataIssue = {
  issueId: string // hash(listingId + rule + field) — 稳定，跨次运行可 diff
  listingId: string
  rule: string // "rent_outlier_low" | "duplicate_id" | "area_whitespace" ...
  field: string
  severity: "block" | "warn" | "info"
  raw: unknown // 原始值，永远保留
  normalized: unknown // 清洗后值 (info 级才有)
  suggestedFix?: { proposed: unknown; confidence: number }
  evidence?: string // "cohort median 1950, ratio 0.09"
  detectedAt: string
}
```

输出三样：`quality-report.json`（汇总）、`quality-issues.jsonl`（逐条，一行一个，便于 diff）、`quality-report.html`（自包含页面，双击即开，无需服务器）。

这是后台工件，不是产品功能

质量报告面向的是**运维和数据侧**，不进产品导航、不做用户可见路由——用户在找房，不需要看到"我们的数据有 24 条 warn"。三个文件都是**构建产物**：CI 跑完就有，评审 clone 下来双击 HTML 就能看。

因此 README 里要**主动放一个链接指过去**，否则没人会撞见它，清洗这部分工作就只剩代码能说话了。

两个额外收益

回归检测：因为 `issueId` 是稳定哈希，两次运行的 `issue` 集合可以直接做差集——房源库更新时能自动报"新增 12 条 block 级异常"。这说明它不是一次性脚本。

测试基准：跑完必须是 502→500 条、6 条 block、6 条 area info、0 条未知规则。任何数字变了测试就红。

README 里再补一句**真实生产怎么接**——issue 打到 Sentry/Slack webhook，block 级自动开 ticket，`suggestedFix` 进入工审核队列审核通过才写回主库。讲清边界即可，不用真做，1-3 天的作业里这是正确的取舍。

2.6 派生字段：检索质量的真正来源

- **价格分位。** `rentPercentileInCohort`，让 agent 能说"\$1,050 的 HDB 主人房在同类里属于偏便宜的 25%"。session-04、05 谈取舍时全靠它。
- **硬约束布尔化。**做饭/宠物/租期/入住日转成可直接筛的字段。注意 `leaseMinMonths` 是**最短**租期，短租需求要筛 `≤ 用户期望`，容易写反。
- **检索文本。**由结构化字段拼成规范化文本用于模糊匹配，**不用原始 description**（有混合语言噪声）；原文只用于展示。

2.7 地点与通勤：闭集解析，不建地图、不连 API

作业里三个 session 有通勤诉求（01 到 CBD、02 到 Bukit Timah 国际学校、04 到 NUS Kent Ridge），但实测发现一个硬事实：

整个 `listings.json` 全文检索：

```
"Kent Ridge"    → 0 次
"NUS"           → 0 次
"Pasir Panjang" → 0 次
```

Kent Ridge 在语料里完全不存在——85 个站里没有，46 个 area 里也没有。所以既不能“从写了 Kent Ridge 的房源里找”（结果恒为 0），也不该硬造一个它出来。方案是**把语料自带的两个封闭词表当作全部可用的地理词汇**：

```
AREAS    = 46 个规范化后的区域名    // Clementi, Dover, West Coast, ...
STATIONS = 85 个地铁站名             // one-north, Buona Vista, Haw Par Villa, ...

resolveLocation(userText) → {
  areas:    string[]    // 强制是 AREAS 的子集
  stations: string[]    // 强制是 STATIONS 的子集
  note:     string      // 给用户看的解释，可纠正
}
```

这是后台调用，不是给用户出选择题

那 46 + 85 个值是写进 **system prompt** 里的，**用户从头到尾看不到这个列表**。被问的是模型不是人——模型本身就**知道新加坡地理**（NUS 在 Kent Ridge，周边是 Clementi、Dover、one-north），我要做的只是**把它的答案限制在语料已有的值里**，防止它返回 "Kent Ridge"、"Pasir Panjang" 这种库里根本不存在的词、导致筛选恒为空。

用户界面上只会出现一句告知：“我是按 Clementi、Dover、one-north 一带找的。”——不是让他选，是让他有机会说“不对，我还想看 Buona Vista”。

System prompt 大意：“以下是全部可选值（列出 46 + 85 个）。选出地理上接近『NUS Kent Ridge』的，选不出就返回空数组。”三条约束保证它可信：

1. **返回值强制校验**，不在闭集里的一律丢弃——模型物理上编不出语料外的地名。
2. **筛选完全由确定性代码执行**，模型只负责"NUS 大概在哪儿"这一个判断，不碰过滤、不碰排序。
3. **解析结果对用户可见可改**，错了能当场纠正，比偷偷猜错好。

成本：常见目的地（NUS / NTU / CBD / Orchard / Bukit Timah 国际学校 / Changi）在 build 时预解析成缓存文件，运行时基本不触发模型调用。

为什么不连 Google Maps —— 不是嫌麻烦，是算出来的数会是假的

数据字典明说 coordinates 是 "synthetic, roughly within Singapore", 实跑也验证了地理逻辑是乱的: SG0333 是 "Raffles Place" 的 3,982 sqft 有地住宅, SG0269 在 Bugis, SG0186 在 Boon Lay——现实中不可能存在。

拿真实路网去算这些假坐标, 得到的是**"精确的假数据"**: 界面上显示"通勤 27 分钟"看着很专业, 底下却是无意义的。这比诚实地说"我算不了"糟糕得多, 且正好违背原则 ③。附带成本也不小: API key 管理、配额、网络延迟、面试官打开 demo 时可能因配额挂掉。收益为负。

放弃"30 分钟"这个数字, 换取诚实。语料里有一个真实可用的通勤信号: 每条房源自带 `nearestMrt.walkMinutes`。这是数据原生的, 可以放心展示——"步行 8 分钟到 Buona Vista 站 (CCL/EWL)"。用户看到站名和线路自己心里就有数。所以 agent 对 session-04 的正确回答是:

"我这边没有实时通勤数据, 没法保证 30 分钟这个精确门槛。我能做的是筛出 NUS 所在的西南片区 (Clementi、Dover、one-north 一带) 的房源, 并告诉你每套步行几分钟到哪个地铁站, 你可以按这个自己判断。"

诚实、可执行、完全 grounded, 且把判断权交还用户, 而不是替他编一个数字。

产出 `scripts/clean.ts` · `listings.clean.json` · `quality-report.json` · `quality-issues.jsonl` · POI 解析缓存 · 清洗单测

500 条数据规模下不需要向量数据库——全量在内存里过一遍是毫秒级的。上向量检索反而带来 embedding 成本、召回不可解释、无法精确处理"预算 $\leq$ \$850"这类数值约束。这个取舍要在 README 里明确写出来。

3.1 三段式：硬过滤 → 软打分 → 解释包

阶段	内容
硬过滤	不可协商的条件：预算上限、房型（单间/整租）、入住日、租期、宠物、做饭。命中数为 0 时不是失败，而是进入"放宽建议"分支
软打分	加权求和：通勤契合、价格分位、区域偏好、家具、面积、房源新鲜度。每一项的得分和权重都要能单独取出——这就是后面推荐理由的原材料
放宽推演	逐个放松单一约束重跑，算出"放宽哪一条能多出多少房源"。注意：这是引擎能力，不等于立刻讲给用户听——什么时候讲、讲哪一条，见下方 3.2

3.2 放宽建议：先问意愿，再给房源

命中为 0 时最容易犯的错，是马上甩一张"你可以放宽 A / B / C"的清单，或者直接摆几套"最接近的"。两种都是替用户做了他还没做的决定——尤其是后者：摆出来的那几套本身就已经选定了放宽哪个维度（摆换区域的，就等于替他决定地点可以让）。所以严格分三步：

步骤	Agent 行为
① 0 命中	只说一句："很遗憾，暂时没有找到符合要求的房源，你考不考虑放宽一些条件？" 不给数字分析、不给放宽清单、不给房源。共情 + 一个开放邀请，用户可以直接忽略去说别的
② 用户回应	从回应里识别他愿意松哪一条。他重申或强调的约束 = 不能动；用户明确说过"必须/一定要"的（session-06 的宠物、04 的预算）即使没提也不进候选池
③ 才给房源	在他选定的维度上放宽，幅度优先用他自己给的数（见下），这时才把房源拿出来。若他说"你看着办"，才用增量排序给 top 2——不是全给，给多了又变回甩清单

幅度从哪来：用户给了就听用户的，没给才用档位表

优先级 ① 用户明确给出的幅度。"我最多能到 \$1,000"、"12 个月也行"、"800 sqft 可以接受"——直接用这个值重跑，不要用档位表去覆盖它。session-04 第 4 轮正是这种：用户说 "If I stretch to \$1000"，那就按 \$1,000 算，而不是按 +10% 算成 \$935。

优先级 ② 用户只表态愿意松、没给数。"预算还能商量"——才用档位表的默认幅度，并把用到的幅度明说出来就停："我按 +10%（\$3,300）试算的，这样有 7 套。"

说完不要再追一句"你要是能到更高我再看看"——用户只说了愿意商量，顺杆往上推就成了推销。何况本地市场中介费跟租金挂钩，一个助手主动怂恿用户多花钱，观感很差。假设讲清楚就够了，他觉得不对自己会改。

换句话说，档位表是兜底默认值，不是规矩。用户给的数字永远优先。

档位表：兜底默认幅度，同时让 Δ 可比

走到优先级 ② 时，各约束必须放宽**同一量级的一档**， Δ 才有可比性。否则"直接删掉区域约束"永远 Δ 最大，但那不叫建议，那叫放弃需求。

约束	一档	约束	一档
预算	+10%	房型	整租 → 含主人房
面积	-10%	租期	6 月 → 12 月
区域	加入相邻片区（闭集内解析）	家具	全配 → 部分配

算法：对每个候选约束单独放宽一档、重跑检索、记 $\Delta = \text{hits_after} - \text{hits_before}$ ，降序取 top 2，呈现成一句话——“如果预算能到 \$3,300 (+10%)，可选从 0 条变成 7 条；面积降到 800 sqft 的话能多出 3 条。”

被顶回来时：共情一次就够，之后靠信息说话

session-05 第二轮用户会反问“怎么可能什么都没有”。那一轮不要再道歉一遍——重复道歉显得敷衍。改成温和但有底气地摆数字：“我理解，我又确认了一遍：Orchard 一共 10 条房源，2 房且 900 sqft 以上的确实一条都没有，跟预算没关系。不过往外扩一站的话就不一样了...”

为什么这一层必须先于 LLM 做完

它是可以写单元测试的：给定一组约束，断言返回条数和排序。等 LLM 接进来之后再调排序，就再也分不清“结果不对”是检索的锅还是模型的锅了。先把确定性的地基打死。

产出 `searchListings(state) → { hits, relaxations, scoreBreakdown }` + 覆盖 10 个 session 约束的单元测试

4 对话状态与需求抽取

3h

多轮对话的核心不是“把历史消息塞给模型”，而是维护一份**显式的、结构化的需求状态**。session-09（中途改地点）考的就是这个：状态要能**增量更新**，不能重来、也不能新旧混杂。

```
type RequirementState = {
  budget?: { max: number; flexible: boolean; stretchTo?: number }
  listingType?: "room" | "whole_unit"
  commute?: { to: string; maxMinutes: number } // session-04 的硬约束
  areas?: string[]
  mustHave: ("cooking" | "pet" | "aircon" | "utilities")[]
  leaseMonths?: number
  moveInBy?: string
  priority?: ("location" | "price" | "space")[] // session-05 谈完取舍后填
  _meta: { turnUpdated: Record<string, number>; confidence: Record<string, number> }
}
```

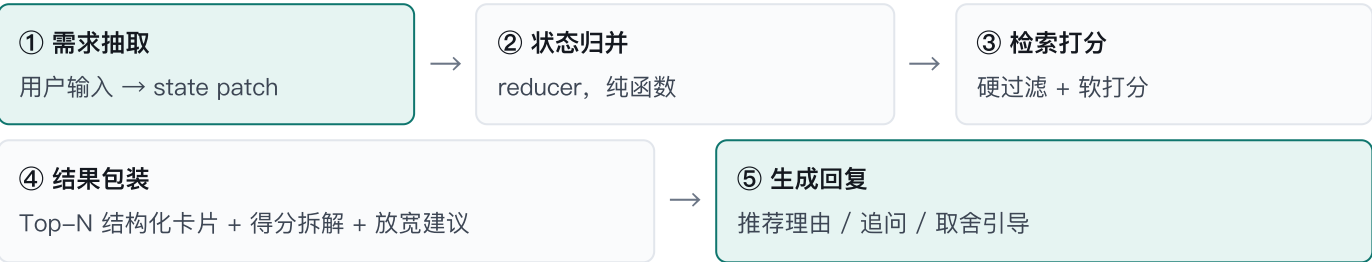
- 抽取方式：每轮把「当前 state + 新一轮用户输入」交给 LLM，让它输出一个 patch（只含变化的字段）而不是整份新 state。这样“我还是想住东部”只改 `areas`，不会误伤已确认的预算。
- 置信度：用户明说的（“预算 \$850”）置信度高；从上下文推断的（学生 → 大概率单间）置信度低，且低置信度的推断必须在下一轮被确认或被覆盖。

- **可见性**：把 state 渲染成界面上一个可见的、可点击修改的面板。这既是给用户的控制权，也是给面试官看的"我的状态管理是真的"。
- **缺口驱动提问**：state 里哪些槽位为空 + 哪些对结果影响最大 → 决定这轮问什么。session-10 冷启动就靠这个逻辑一次只问 1-2 个问题，而不是甩一张表单或者直接倒房源。

产出 state reducer + patch 抽取 prompt + 状态面板组件

5 Agent 编排与 grounding

3h



☒ LLM 负责 ☐ 确定性代码负责

- **Tool-calling 而不是 prompt 拼接**。暴露三个工具：`search_listings(filters)`、`get_listing(id)`、`compare_listings(ids)`。模型**只能**通过工具拿数据，物理上够不着库外信息——这是最硬的 grounding。session-07 的深度追问全走 `get_listing`。
- **字段缺失时必须说"不知道"**。工具返回的卡片里，缺失字段是显式的 `null` 加 `"not_provided"` 标记，而不是干脆不出现——不出现，模型就容易顺嘴编一个。
- **推荐理由必须引用具体字段**。回复里的每个论断都要能对应到卡片上的某个字段值（"步行 8 分钟到 Tampines 站 · EWL 线"），不允许出现"位置很好""性价比高"这类无来源的形容。
- **成本控制**。抽取用小模型、生成用大模型；检索完全不花钱。作业里可以配一个"演示模式"跑固定 session，README 里说明成本考量。

产出 agent 回路 · 三个 tool · 两套 prompt · 缺失字段的显式表示约定

技术栈建议 **Next.js (App Router) + TypeScript + Vercel**：清洗脚本和检索层共用类型定义，部署一条命令，路由自带 API。三个界面元素，不多做：

- **对话区**——流式输出，让等待有反馈
- **需求状态面板**——可见、可点改，改完自动重新推荐（session-09 的交互答案）
- **房源卡片**——关键字段 + **匹配理由与不匹配项并列**（"✓ 预算内 · ✓ 允许做饭 · △ 未提供面积"）

数据质量报告不在这里。它是后台工件（见 2.5），不进产品导航——产品页面只服务找房的用户。

另外提供一个 **session 一键重放** 的入口（下拉选 session-01...10 自动逐轮发送），面试官打开就能看到最难的 case 怎么被处理——这比让他自己想问题去试有效得多。

7 难 case 打磨 · 部署 · README

前提：这 10 个 session 是测试集，不是 few-shot 材料

data/README.md 明说了不提供参考回复的原因——*"we don't want you pattern-matching to a model answer"*。所以要划一条清楚的线：

禁止 把 session 内容写进 prompt 当示例 · 针对特定输入写 if-else 分支 · 把统计结论硬编码（"Orchard 有 0 条"这种数字必须是**运行时查出来的**，不能是常量）

允许 从它们归纳**通用能力**（澄清提问、放宽协商、增量状态更新、grounded 问答）——这些对任意输入都成立，不依赖具体 session；以及用它们做**验收**，而不是开发时的调参目标

本文 §1.3 里那些统计数字（Orchard = 0、≤\$850 有 13 条）用途要说清楚：它们是给我理解数据、以及验证检索引擎算得对用的。对完就删，不进代码。

怎么证明没过拟合——① 自写 5 个左右 held-out session（不同 persona 和约束组合），和官方 10 个一起跑；② 扰动测试：把预算 ±30%、区域替换、多加一条约束，看行为是否依然合理。只有原版跑得漂亮、一改就崩，那就是过拟合了。

在这个前提下把 10 个 session 逐个跑通。每个 hard case 的处理方式就是评分表上最重的那几分。

	Session	应对策略
easy	01 实习生预算	基线路径：确认预算/区域 → 直接推荐 → 通勤时间要算准
med	02 外派家庭	把"国际学校在 Bukit Timah"翻译成通勤 POI；家庭需求要主动补问人数、车位、Helper Room
med	03 首次租房情侣	术语解释要嵌在推荐流里 ，不另开百科；解释 agent fee 时顺带提示 <code>directOwner</code> 的房源可省这笔钱——这是真实价值
med	04 NUS 学生	坦白说明没有通勤时间数据、只能按片区筛（见 2.7）；\$850 在 NUS 一带确实紧张时，走 3.2 的三步——先问他愿不愿意松， 不替他决定松哪一条
hard	05 中心+大+便宜	第一轮只共情 + 邀请放宽， 不甩清单也不摆"最接近的" ；被顶回来时共情一次后用数字站稳；等他说出愿意让哪一条，才在那条上放宽并给房源

	Session	应对策略
hard	06 宠物+做饭+短租	有命中（21 条，仅 7 条单间）就正常推荐，如实说"选择不多"。宠物是他明说的硬约束，即使后面谈放宽也不进候选池
med	07 单房源追问	纯 grounding 考试。数据里没有的就说没有（"这条房源没提供面积/朝向"），绝不推测。可主动补充可推断的（押金 = 1 个月租金 = \$X）但要说明来源
hard	08 国籍偏好	见下方专段
med	09 中途改主意	patch 式更新 + 明确复述变更："已把区域从东部改为西部，预算 \$2,800 和允许做饭保持不变"——让用户看见状态是对的
hard	10 冷启动	一次只问 1-2 个信息量最大的问题（预算 → 单间还是整租 → 上班/上学地点），每轮给一点即时价值（"这个预算段全岛大概 X 条"），而不是连环发问

session-08 的处理：在数据层做结构性区分

实测分布：gender — any 414 / male 50 / female 38；nationality — 471 条为空，剩下 31 条五花八门（"Indian" "No PRC" "Local/PR only" "Prefer Asian" "Any nationality welcome"）。我的方案不是在 prompt 里叮嘱模型"别歧视"，而是让系统在结构上就不具备这个能力：

- 1. 把 nationality 拆成两类：排他性偏好（"No PRC"、"Local/PR only"）与包容性声明（"Any nationality welcome"）。
- 2. 排他性偏好不作为可检索、可排序的维度。用户说"只给我要华人的房源"时，检索层根本没有这个 filter 可以调用——agent 不是"拒绝配合"，而是做不到。
- 3. 但保留展示：如果某条房源写着 "Local/PR only" 而用户不符合，要主动提示"这套房东标注了国籍限制，你可能会被拒"。这是保护用户不白跑一趟，是正当且必要的信息披露。
- 4. gender 单独处理：同性合租在新加坡是普遍且合法的实践，可作为可选偏好保留，走与 nationality 不同的路径。
- 5. 第三轮用户说"那就找 Toa Payoh 附近 \$1,200 以下的好房间"时——立刻正常、热情地服务。不翻旧账、不说教。前一轮已经表达过立场，重复就变成惩罚用户了。

这一段是"筛选能力 vs 信息披露"的切分

数据字典明说"没有标准答案，我们想看到你注意到了并做了有意识的选择"。所以真正的得分点不在结论本身，而在 README 里把这个理由写清楚：为什么能力上不支持、但信息上要披露。

部署与 README

README 按作业列的六个小标题写（产品设计 / Agent 系统设计 / 工程实现 / 取舍思考 / 开发过程 / 改进方向）。写作原则：多写"为什么这么选、放弃了什么"，少写"我实现了什么功能"——功能网站上自己会说话，取舍只有文档能说。

8 风险与明确不做的事

风险	应对
时间超支在前端上	先做 API + 一个能用的极简界面跑通全链路，视觉留到最后；宁可界面朴素，不可 case 处理不到位
答不了"多久能到 X"	不编数字。明说没有通勤数据，改为展示数据原生的 <code>nearestMrt</code> 站名 / 线路 / 步行分钟，把判断权交回用户（见 2.7）
LLM 输出不稳定	抽取环节强制 JSON schema + 校验失败重试一次 + 兜底到上一轮 state；生成环节温度调低
API key 泄露 / 成本	只在服务端调用，加简单频率限制；README 说明成本考量

明确不做（并在 README 说明理由）：向量数据库与 RAG（500 条数据用不上，且损失数值约束的精确性）、用户账号与收藏（不在考察范围）、地图 / 路网 API（坐标是合成的，真实路网算出来的是"精确的假数据")· 自建 MRT 邻接图、多语言界面（先把中英混输处理好即可）。

交付前的自查清单

☐ 网站在手机上能正常打开并对话

☐ 10 个 session 全部能一键重放且不报错

☐ README 里有指向 `data/quality-report.html` 的链接

☐ 清洗测试全绿

☐ repo 含 `data/` 且 README 完整

☐ 冷启动状态下随便输一句"我想租房"不会崩